

Article

An Open-Source Tree Bucking Optimizer Based on Dynamic Programming

Caroline Bennemann ^{1,2,*}, Jean-Martin Lussier ² and Eric R. Labelle ¹ 

¹ Department of Wood and Forest Sciences, Faculty of Forestry, Geography and Geomatics, Université Laval, 2405 rue de la Terrasse, Québec, QC G1V 0A6, Canada; eric.labelle@sbf.ulaval.ca

² Canadian Forest Service, Natural Resources Canada, 1055 rue du Peps, Québec, QC G1V 4C7, Canada; jean-martin.lussier@nrcan-rncan.gc.ca

* Correspondence: caroline.bennemann.1@ulaval.ca

Abstract: Bucking optimization was shown to generate high gains in volume and value recovery in harvesting operations. Several bucking optimizers have been developed between the 1960s and early 2000s but none of those programs were available in an open-source and easily modifiable format by users. Therefore, this paper presents BuckR, an open access bucking optimizer at tree-level, programmed in R. The objective function is the maximization of values for each study tree, but it can be modified depending on the needs of users. BuckR is based on a well-known dynamic programming algorithm and requires tree data and product specifications with prices as the inputs. One of the output files produced is the log sequence, generating the highest value for each study tree. Through the open access code, the developed bucking optimizer will facilitate future research and applications in the field of value maximization.

Keywords: optimization; value maximization; product recovery; volume recovery

1. Introduction

Forest operations are essential in sustainable forestry as they allow the harvest of wood and thus reflect the core moment when silvicultural investments are returned to forest owners and managers [1]. An astute use of wood is particularly important as its production requires several decades, especially when the goal is to obtain sawlog dimensions.

All decisions taken during the harvesting phase have irreversible consequences on the final volume and value recovery and cannot be counterbalanced in subsequent steps [2,3]. Bucking, the task of cross-cutting stems into logs of varying assortments [4], is described as a pivotal decision [5] with a high impact on value creation [6] during harvest operations. During bucking, several aspects, such as the tree species, stem quality, dimensions [7], final use of wood products [4], and transport to the mill [8], need to be considered to meet product demands and maximize value recovery.

Research on bucking optimization has been conducted since the 1960s [3], both at tree-level (for example [4]) and stand-level (for example [9,10]), by using different mathematical approaches, such as linear programming, dynamic programming, network analysis, shortest path algorithms, or heuristic approaches [11]. In the late 1970s, the VISION optimizer was developed to optimize softwood bucking and showed several million dollars of additional benefit through its application [5,12]. Liu [13] gives an overview of research on different published bucking algorithms [14].

Bucking optimization is now widely available and used in modern cut-to-length (CTL) harvesters for the harvest and merchandizing of softwood [15,16]. In Eastern Canada, three-quarters of harvesting operations are performed using harvesters and the CTL method [17],



Academic Editors: Kristina Klarić,
Andreja Pirc Barčič and
Nikolay Neykov

Received: 1 April 2025

Revised: 24 April 2025

Accepted: 2 May 2025

Published: 6 May 2025

Citation: Bennemann, C.; Lussier, J.-M.; Labelle, E.R. An Open-Source Tree Bucking Optimizer Based on Dynamic Programming. *Forests* **2025**, *16*, 780. <https://doi.org/10.3390/f16050780>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

where bucking is performed directly in the stand. The use of harvesters is more common in softwoods but has also been increasing in hardwoods [18]. Harvesters, initially developed to reduce the costs of forest operations and replace costly manpower [19], lead to a high increase in productivity [17] and work safety [20], addressing ergonomic challenges. Labelle and Huß [21] reported an increase in productivity of up to 17% when automatically bucking Norway spruce (*Picea abies* [L.] Karst.) as opposed to quality bucking performed by the operator, a result supported by Kärhä et al. [22] in Scandinavia and Boston, and Murphy [23] in the Southeastern United States.

Hardwood bucking is generally more challenging because of complex tree architectures [24] and the wide range of industrial utilizations, from low-value pulp products and bioenergy to high-end veneer and furniture products. This leads to a more diverse and complex list of log specifications, in addition to a wider range of log values than a typical softwood bucking operation. Alas, automated bucking with CTL harvesters does not consider value-conditioning stem defects or damages, such as knots, rot, or crooks [22], and, therefore, generally cannot be used in hardwoods, yet. However, it is hypothesized that the optimization of bucking in hardwood can lead to higher potential value recovery with better utilization of the resource than for softwood because hardwood tree architecture leads to complex bucking decisions [8] and a higher range of log values.

Early work in hardwood bucking optimization by Pickens et al. [25] involved a bucking simulator called HW-Buck that used dynamic programming. It was specifically tailored for hardwoods and their defects, assessing the dimensions and curvature of the main stems. This software was developed prior to the advent of harvesters. It was designed as a serious game to train buckers and help them understand the impact of different quality factors and markets on value recovery. The application of HW-Buck showed potential gains of 39%–55% in value recovery, depending on the tree species [8].

More recently, a bucking optimizer called NECO was developed for both softwood and hardwood species found in Northeastern China using network analysis [7]. CAHO, another bucking optimizer, was developed for Central Appalachian Hardwoods, including five oak species (*Quercus* spp.) and yellow poplar (*Liriodendron tulipifera*) [26]. CAHO is based on dynamic programming and is governed by log grading rules used in the study area. Table 1 provides an overview of different optimizers developed and used for studies on hardwood recovery at tree-level.

Table 1. Overview of bucking optimizers developed for hardwoods.

Program Name	HW-Buck	HW-Buck 2	NECO	CAHO
Related publication	Pickens et al., 1991 [25]	Pickens et al., 1991 [25]	Wang et al., 2004 [7]	Wang et al., 2009 [26]
Geographic region or forest	Northern Hardwoods, Northeastern USA	Northern Hardwoods, Northeastern USA	Northeastern China	Central Appalachian Forests, Eastern USA
Availability	Online https://hardwoodbucking.mtu.edu/downloads.htm (accessed on 5 May 2025)	Through personal communication with first author	Not online	Through personal communication with first author
Inclusion of quality factors	Yes	Yes	Yes	Yes
Mathematical approach	Dynamic programming	Dynamic programming	Network analysis	Dynamic programming
Unit system	Imperial	Imperial	Metric	Metric
Log scaling rules	Scribner Decimal C and International ¼-Inch Log Rule	Scribner Decimal C and International ¼-Inch Log Rule	According to China Wood Standardization Committee	Doyle International ¼-Inch Log Rule
Goal of the program	Train buckers by “playing”	Train buckers by “playing”	Evaluation of actual bucking	Evaluate actual bucking and train operators
Customizable	No	Partly (markets (with restrictions) but not log scaling rules)	Unknown	Partly

Even though several bucking optimizers were used in recent decades [13], none of those are available as an open-source program, and none were easily customizable, which hinders further research in this field. The objective of this paper is therefore to present an open-source bucking optimization program called BuckR, based on a well-known bucking

optimization algorithm. The program is published as a script rather than a stand-alone application or package, so all parts of the program can be adapted to the needs of users for research and application purposes. The program is freely available, with a working example in the Supplementary Material.

2. Development and Implementation

2.1. Dynamic Programming

Several mathematical programming techniques have been used to solve optimal bucking problems [13]. First described in the middle of the 20th century [27], dynamic programming was used by Pnevmticos and Mann [2] specifically for the bucking problem, and allowed the identification of optimal solutions. This approach is commonly used in other contexts of optimization [28] and recent studies on tree bucking [29]. One of the main advantages of this technique is the subdivision of the overall problem into stages [12], thus reducing the size of the problem. For the bucking problem, each stage corresponds to a log, starting from the cross-cut at the stump (first log, second log, etc.). For each stage, the best solution is assessed, memorized, and then used to consider the next stage. An educational example of the application of dynamic programming for the bucking problem was proposed by Dykstra [30].

2.2. Implementation in R and Objective Function Definition

The programming language R [31], version 4.4.1 (14 June 2024), was chosen to implement the bucking optimizer using dynamic programming. This language was selected for its broad availability due to it being open-source and because it is widely used in forest sciences.

The goal of BuckR was to maximize tree-level value by the sum of the value of the produced logs (“buck-to-value”). The program was designed to sequentially process tree stems. The tree-level value was calculated as the sum of the value of all merchantable logs. The current version BuckR is designed to deal with one tree species and one type of product specifications at the same time. The application to multiple species requires separate runs of the program. The workflow used by BuckR is represented in Figure 1 and described in detail in the subsequent sections.

2.3. Optimization Algorithm

2.3.1. Input Files

Two CSV comma-delimited text files are required as the input for BuckR (step 1.4 in Figure 1). The first file contains stem profile information on tree data, i.e., diameters in millimeters measured at regular intervals along the longitudinal axis of the main stem from the first cross-cut at the bottom of the stem. The 1 cm interval was chosen because it is the resolution at which the log lengths are set in the industry. The choice of units was based on the StanForD 2010 standards [32]. The second file is for product specifications with the associated prices for the desired logs. Column names, a description, and the data type of both input files are listed in Table 2. Even though the current version of BuckR is designed to consider only one tree species at a time, a column with species is found in the input files to ensure that the tree data and product specifications fit together.

Stem profiles can be extracted, for example, from on-board computers of harvesters or generated from tree inventory data using tree taper equations.

Product specifications can be modified according to the operational context. The variable SEDmax may not be foreseen for all the items in the original product specifications. In this case, the inclusion of a value greater than the largest diameter of the largest study tree is necessary. Each length needs to cover the same range of diameters to avoid false

classifications. For example, the log grading rules proposed by Dykstra [30], which are used for the validation of this tool, do not cover logs less than 450 cm in length and more than 500 mm in diameter. A graphical representation of log grades with associated prices is given in Figure 2A. Figure 2B shows the updated grading rules with an additional category in which the value of zero is attributed to logs. Within the optimization process, this means BuckR is allowed to generate such logs, but as their value is null, they should not be selected when maximizing the value per stem.

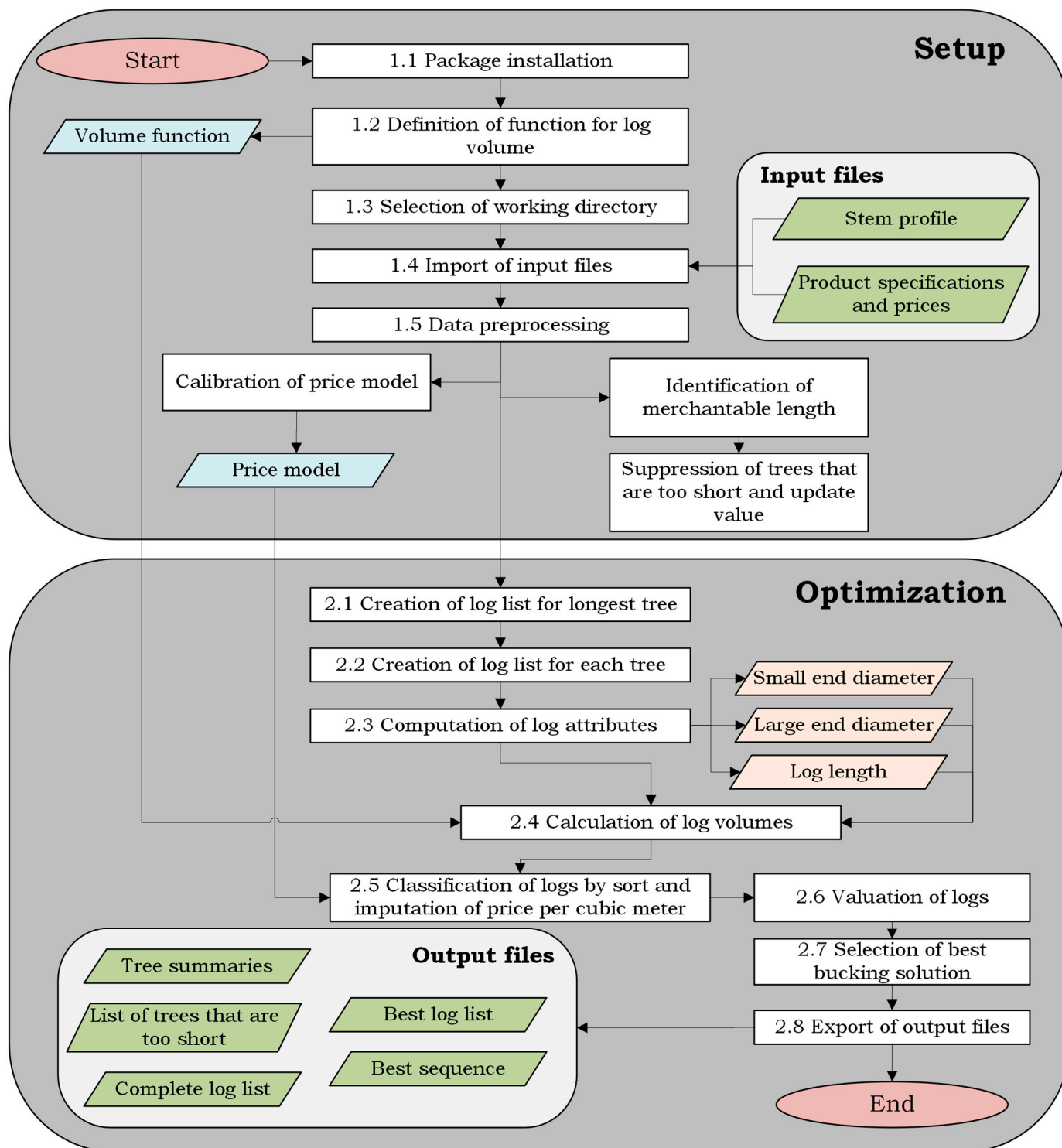
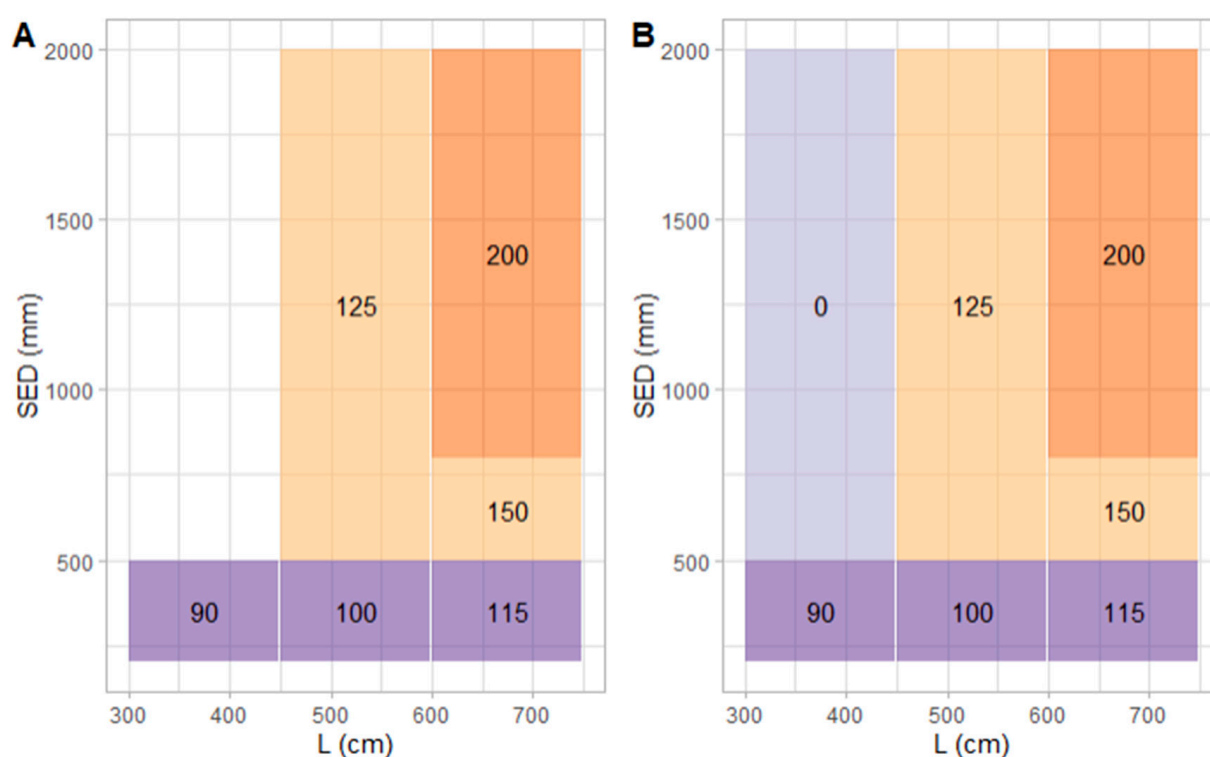


Figure 1. Workflow of bucking optimizer BuckR.

Table 2. Name, description, and data type of data included as tree data and product specifications into BuckR.

Input File	Name of the Variable	Description	Data Type in R
Tree data	Tree.No	Sequential number of trees (1 to x)	Factor
	ID.Tree	Alphanumeric variable for identification of study trees	Factor
	SP	Tree species	Factor
	LG	Vertical position from the first cross-cut in 1 cm increments (cm)	Numeric
	DG	Diameter for each vertical position (mm)	Numeric
	DBH	DBH of study tree (cm)	Numeric
	HT	Total height of study tree (m)	Numeric
Product specifications	ProductID	Sequential number of products (1 to x)	Factor
	SP	Tree species	Factor
	L	Requested log length (including overlength, cm)	Numeric
	SEDmin	Minimum diameter of product (mm)	Numeric
	SEDmax	Maximum diameter of product (mm)	Numeric
	VAM	Value of product (unit per cubic meter)	Numeric
	cat	Category of product ("saw", "pulp" or "waste")	Factor

**Figure 2.** Log grades proposed by Dykstra [30], based on log length and small-end diameter. (A) shows the initial foreseen grades; (B) shows the version updated for BuckR. Different colors represent grades proposed by Dykstra [30], and numbers represent the value associated.

In certain instances, the log length in product specifications may not be implicit, e.g., when random lengths are allowed. In this case, all possible log lengths need to be hardcoded to give BuckR the possibility of applying different lengths (typically in 30 cm increments). All those hardcoded lengths need to have the same value of product (VAM) associated. A filter is applied to ensure the merchantable length is longer than or equal to the shortest product length. If this condition is not met, the particular study tree is excluded from the optimization.

2.3.2. Classification Model

Product specifications and prices are used to create a tree classification model with the rpart package [33] to avoid hardcoding a cascading suite of conditions (Figure 1,

step 1.5). The model predicts the ProductID based on the log length (L) and small-end diameter (SED). Prior to model calibration, a training dataset is generated by creating a combination of values with given increments. For each ProductID, the SED is a sequence of diameters ranging from SEDmin to SEDmax. The classification model and the optimization algorithm can be modified later when specifications include log quality factors. As the classification model is used to label the generated products, an accuracy of 1 is targeted. If the specifications are explicit, this should be the case. The obtained classification model is applied later in the process (Figure 1, steps 2.1 and 2.2).

2.3.3. Creation of Log Combinations

The baseline of the stem length is set at the first cut (stump cut). The merchantable length of each tree is calculated based on the smallest small-end diameter (SEDmin) throughout all product specifications available. Therefore, logs can be generated within the main stem located between the stump and the position of the minimum merchantable diameter.

The number of stages to consider for a group of trees is based on the maximum number of logs that can be bucked with the longest tree. For example, based on the example of Dykstra [30], the study tree has a merchantable stem length of 15 m, and the shortest possible log is 300 cm. This means this tree can generate up to five logs; thus, optimization has five stages.

The possible combinations of logs are determined stage by stage. For each stage, we consider the combination of all possible log lengths from the price list to all possible cumulative log lengths from the previous stage. This list of possible logs per stage is then copied for each tree in the input file, and the log list is adjusted to consider the merchantable length of each tree in the input file.

In a subsequent step, for each tree, large- and small-end diameters for each log are imputed from the stem profile table (Figure 1, step 2.3). These diameters and lengths are used to calculate the volume of each log using the Smalian formula [34], and these volumes are therefore expressed in cubic meters (Figure 1, step 2.4).

2.3.4. Classification of Logs and Attribution of Value

The classification model is applied to the previously generated log list for each tree (Figure 1, step 2.6). Once classification is complete, a value per cubic meter (VAM in the input file) can be applied to the log volume and thus generate the value of each log for all possible logs.

2.3.5. Selection of the Best Bucking Solution

For each stage, the value of a log sequence (defined by a log sort and cumulative log length from the previous stage) is equal to the value of the log added at this stage, plus the maximum value of the log combination for the cumulative log length from the former stages ($f_i^*(s)$ from Equation 10.6 in Dykstra [30]).

The best bucking solution is the one that shows the maximum value of $f_i^*(s)$ for all stages. The optimal sequence of logs leading to this solution is then identified in a backward procedure (Figure 1, step 2.7).

2.3.6. Output

BuckR generates five files as output files (Figure 1, step 2.8). The most important one contains the best sequence of logs for each tree, thus describing the bucking solution that maximizes value recovery for each tree. This file is a subset of the file with all possible logs for each tree. Moreover, a summary of the study trees is generated, including ID.Tree and variables such as DBH and tree height. Other outputs include the list of too-short trees

and the list of possible log lengths by cumulative log lengths for each stage (“complete log list”).

3. Validation

To validate BuckR, the example proposed by Dykstra [30] when explaining dynamic programming was used. A tree with a 2000 mm diameter at the stump and a merchantable length of 1500 cm is bucked. The tree profile is assumed to be simply modeled by a reduction in stem diameter of 12 cm per meter length (1.2 mm/cm). Logs foreseen by specifications were 300 cm, 450 cm, and 600 cm in length. Log prices varied according to the log length and small-end diameter (Table 3). Modifications made for this validation include, for example, the use of the volume formula described in the example instead of the use of the Smalian volume.

Table 3. Price list used to validate BuckR, adapted from Dykstra [30].

Product ID	Log Length (cm)	Minimum Small-End Diameter (mm)	Maximum Small-End Diameter (mm)	Value per Cubic Meter
1	300	200	499	90
2	300	500	2000	0
3	450	200	499	100
4	45	500	2000	125
5	600	200	499	115
6	600	500	799	150
7	600	800	2000	200

BuckR generated the same bucking solution and therefore the same value recovery, as described by Dykstra [30] for the study tree. Thus, BuckR is considered to be operating as intended. The running time, starting once all necessary packages are loaded, was approximately 2.5 s. This was tested on a system running Windows 11 Enterprise (64-bit), equipped with a 12th-Gen Intel® Core™ i5-1245U processor. The system was configured with 16 GB of RAM and utilized the default system settings without any specific performance optimizations or load balancing.

4. Discussion

BuckR is a functional open-source tree bucking optimizer that can be used for several optimization problems. In this case, BuckR is programmed with dynamic programming, but several other mathematical approaches may have led to comparable results [13]. The objective function of BuckR is to maximize tree value, but slight modifications allow the maximization of volume recovery. We were able to reproduce the example provided by Dykstra [30], which can be considered as a validation of the algorithm. The main value of BuckR rests in its open-source code that can be modified and adapted for research or operational application, based on a trusted algorithm.

BuckR may be modified depending on the needs of users, making it a helpful tool for understanding the volume and value recovery and paving the way for a sustainable use of wooden resources.

- The volume calculation is based on the volume formula proposed by Dykstra [30], but it can be adapted to others, depending on the jurisdiction and regulation. The volume determination chosen was shown to impact the volume results [35]. This may affect the bucking solution found by the algorithm, depending on the log scaling formula or table used.
- Sometimes, mill specifications allow for random lengths rather than fixed lengths for logs: in this case, with BuckR, it is mandatory to code a series of products with different lengths of a fixed interval and a common value per cubic meter.

- The code can be adapted to process more than one species (in such cases, we recommend the calibration of a log classifier for each species).
- BuckR can be modified to consider stem defects and sweep criteria that are commonly used for the bucking of high-quality hardwoods or softwoods and is shown to impact bucking [8].
- The application of bucking optimization in harvesters implies decision-making with partial information about the stem profile: for each tree, an early bucking solution is calculated based on the profile measured at the bottom of the stem, which is later revised and corrected with supplemental data on the tree shape. This situation is not covered by the current version of our algorithm.
- Finally, a more demanding upgrade would imply the bucking of trees with large, marketable branches, a common issue in hardwoods [24].

This algorithm can be applied to various areas of research, including assessing the performance of bucking based on human decisions; the development of bucking methods specific to hardwoods; operational applications, such as finalizing price lists for harvesters; and forecasting the distribution of logs for operation planning from specific cut blocks. We hope that this open access code, available in the Supplementary Materials of this article, will accelerate research and foster collaboration in forest science.

5. Conclusions

BuckR (version 21 March 2025), which is freely accessible in the Supplementary Materials or through GitHub, can be taken as a base to develop a more user-friendly program tailored to the needs of end users by analysts or IT service providers. Moreover, this open-source software can enable further research to (1) evaluate the performance of bucking operations, (2) update predictive models used in forest management, and (3) adapt bucking algorithms to more complex bucking problems such as hardwoods, which could later be included in harvester OBCs.

Supplementary Materials: BuckR (version 21 March 2025) is accessible through Supplementary Material. It can be downloaded at: <https://www.mdpi.com/article/10.3390/f16050780/s1>, references [30,32] are cited in the Supplementary Material.

Author Contributions: Conceptualization, C.B., J.-M.L. and E.R.L.; software, C.B. and J.-M.L.; validation, C.B., J.-M.L. and E.R.L.; formal analysis, C.B.; data curation, C.B.; writing—original draft preparation, C.B.; writing—review and editing, C.B., J.-M.L. and E.R.L.; visualization, C.B.; supervision, E.R.L. and J.-M.L.; funding acquisition, E.R.L. and J.-M.L. All authors have read and agreed to the published version of the manuscript.

Funding: Funding for this study was generously provided by Mitacs Canada (grant number IT18574) with the Northern Hardwoods Research Institute (NHRI) as project partner and co-funder as well as the University Laval, Natural Resources Canada, and FPInnovations.

Data Availability Statement: The code programmed and the input files for the working example for BuckR (version 21 March 2025) are available in the Supplementary Materials and on GitHub with the following link <https://github.com/CWFC-CCFB/BuckR> (accessed 5 May 2025). Within this document, the dataset used to validate BuckR, based on the example given by Dykstra [30], is available. BuckR is released under a LGPL_2.1 license.

Acknowledgments: The authors want to thank Jim Pickens for his availability and help in untangling bucking optimizers and for being available despite being retired. We also want to thank Mathieu Fortin and all other persons contacted during the process of developing BuckR.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Marshall, H.D. Log Merchandizing Model Used in Mechanical Harvesting. In *Handbook of Operations Research in Natural Resources*; Springer: Boston, MA, USA, 2007; pp. 379–389.
- Pnevmaticos, S.M.; Mann, S.H. Dynamic Programming in Tree Bucking. *For. Prod. J.* **1972**, *22*, 26–30.
- Kivinen, V.-P. A Genetic Algorithm Approach to Tree Bucking Optimization. *For. Sci.* **2004**, *50*, 696–710. [[CrossRef](#)]
- Faaland, B.; Briggs, D. Log Bucking and Lumber Manufacturing Using Dynamic Programming. *Manag. Sci.* **1984**, *30*, 245–458. [[CrossRef](#)]
- Lembersky, M.R.; Chi, U.H. Weyerhaeuser Decision Simulator Improves Timber Profits. *Interfaces* **1986**, *16*, 6–15. [[CrossRef](#)]
- Murphy, G.E. An Economic Analysis of Final Log Manufacturing Locations in the Steep Terrain Radiata Pine Plantations of New Zealand. Ph.D. Dissertation, Oregon State University, Corvallis, OR, USA, 1987.
- Wang, J.; LeDoux, C.B.; McNeel, J. Optimal Tree-Stem Bucking of Northeastern Species of China. *For. Prod. J.* **2004**, *54*, 45–52.
- Pickens, J.B.; Lee, A.; Lyon, G.W. Optimal Bucking of Northern Hardwoods. *North. J. Appl. For.* **1992**, *9*, 149–152. [[CrossRef](#)]
- Sessions, J.; Garland, J. Olsen Testing Computer-Aided Bucking at the Stump. *J. For.* **1989**, *87*, 43–46.
- Laroze, A.J. Development and Comparison of Stand-Level Bucking Optimization Methods. Ph.D. Dissertation, Oregon State University, Corvallis, OR, USA, 1994.
- Laroze, A.J.; Greber, B.J. Using Tabu Search To Generate Stand-Level, Rule-Based Bucking Patterns. *For. Sci.* **1997**, *43*, 157–169. [[CrossRef](#)]
- Rust, J. Dynamic Programming. In *The New Palgrave Dictionary of Economics*; Macmillan Publishers Ltd., Ed.; Palgrave Macmillan UK: London, UK, 2018; pp. 3133–3158. ISBN 978-1-349-95188-8.
- Liu, J. Optimal Bucking Hardwood Species in Central Appalachia. Master's Thesis, West Virginia University, Morgantown, WV, USA, 2006.
- Olivera, A.; Visser, R. Using the Harvester On-Board Computer Capability to Move towards Precision Forestry. *NZ J. For.* **2016**, *60*, 3–7.
- Uusitalo, J. *Introduction to Forest Operations and Technology*; JVP Forest Systems Oy: Tampere, Finland, 2010; ISBN 978-952-92-5269-5.
- Palander, T.; Mutanen, J.; Kärhä, K.; Sorsa, J.-A.; Räsänen, T. Multiple-Criteria Stem Bucking (*Picea abies* L. Karst.) for Maximizing Monetary Value of Timber Trade. *Int. J. Adv. Eng. Res. Sci.* **2018**, *5*, 183–191. [[CrossRef](#)]
- Lundbäck, M.; Häggström, C.; Nordfjell, T. Worldwide Trends in Methods for Harvesting and Extracting Industrial Roundwood. *Int. J. For. Eng.* **2021**, *32*, 202–215. [[CrossRef](#)]
- Labelle, E.R.; Breinig, L.; Sycheva, E. Exploring the Use of Harvesters in Large-Diameter Hardwood-Dominated Stands. *Forests* **2018**, *9*, 424. [[CrossRef](#)]
- Silversides, C.R. Mechanized Forestry, World War II to the Present. *For. Chron.* **1984**, *60*, 231–235. [[CrossRef](#)]
- Gellerstedt, S.; Dahlin, B. Cut-To-Length: The Next Decade. *J. For. Eng.* **1999**, *10*, 17–24.
- Labelle, E.R.; Huß, L. Creation of Value through a Harvester On-Board Bucking Optimization System Operated in a Spruce Stand. *Silva Fenn.* **2018**, *52*. [[CrossRef](#)]
- Kärhä, K.; Änäkälä, J.; Hakonen, O.; Palander, T.; Sorsa, J.-A.; Räsänen, T.; Moilanen, T. Analyzing the Antecedents and Consequences of Manual Log Bucking in Mechanized Wood Harvesting. *Mech. Mater. Sci. Eng.* **2017**, *12*, 1–15.
- Boston, K.; Murphy, G.E. Value Recovery from Two Mechanized Bucking Operations in the Southeastern United States. *South. J. Appl. For.* **2003**, *27*, 259–263. [[CrossRef](#)]
- Millet, J.; Oldemanet, A.A.; Back, F.; Guénette, S. *The Architecture of Trees in Temperate Regions: Its History, Concepts, and Uses*; Jeanne Millet: Montréal, QC, Canada, 2025; ISBN 978-2-9820900-4-0.
- Pickens, J.B.; Lyon, G.W.; Lee, A.; Frayer, W.E. HW-Buck: A Computerized Hardwood Bucking Decision Simulator. In Proceedings of the General Technical Report SE-74; U.S. Department of Agriculture, Forest Service: Charleston, SC, USA, 1991; pp. 213–216.
- Wang, J.; Liu, J.; LeDoux, C.B. A Three-Dimensional Bucking System for Optimal Bucking of Central Appalachian Hardwoods. *Int. J. For. Eng.* **2009**, *20*, 10. [[CrossRef](#)]
- Bellman, R. On the Theory of Dynamic Programming. *Proc. Natl. Acad. Sci. USA* **1952**, *38*, 716–719. [[CrossRef](#)]
- Eddy, S.R. What Is Dynamic Programming? *Nat. Biotechnol.* **2004**, *22*, 909–910. [[CrossRef](#)]
- Prendes, C.; Acuna, M.; Canga, E.; Ordoñez, C.; Cabo, C. Optimal Bucking of Stems from Terrestrial Laser Scanning Data to Maximize Forest Value. *Scand. J. For. Res.* **2023**, *38*, 174–188. [[CrossRef](#)]
- Dykstra, D.P. Chapter Ten - Dynamic Programming. In *Mathematical Programming*. In *Mathematical Programming for Natural Resources Management*; McGraw-Hill Series on Computer Communications: New York, NY, USA, 1984; pp. 287–310.
- R Core Team. *R: A Language and Environment for Statistical Computing*; R Foundation for Statistical Computing: Vienna, Austria, 2021.
- Skogforsk StandForD 2010-Modern Communication with Forest Machines. Available online: https://www.skogforsk.se/cd_20210625001813/contentassets/1a68cdce4af1462ead048b7a5ef1cc06/stanford2010-info-english-2021.zip (accessed on 5 May 2025).

33. Therneau, T.; Atkinson, B. rpart: Recursive Partitioning and Regression Trees. R Package Version 4.1.21. Available online: <https://CRAN.R-project.org/package=rpart> (accessed on 5 May 2025).
34. Kershaw, J.A.; Ducey, M.J.; Beers, T.W.; Husch, B. *Forest Mensuration*, 5th ed.; Wiley/Blackwell: Chichester, UK; Hoboken, NJ, USA, 2017; ISBN 978-1-118-90203-5.
35. Hohmann, F.; Ligocki, A.; Frerichs, L. Harvester Measuring System for Trunk Volume Determination: Comparison with the Real Trunk Volume and Applicability in the Forest Industry. *Bull. Transilv. Univ. Braşov Spec. Issue* **2017**, *10*, 27–34.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.